AI FPGA Engineer: A Reproducible, Toolchain-Backed Benchmark for LLM-Generated VHDL

Clara Ureche
BEng (Hons) Electrical and Electronic Engineering, Lancaster University

Report type	Research report
Status	Research software released (v1.0); benchmark dataset public; preprint available
Outputs	Code: github.com/urecheM/ai-fpga-engineer. Dataset: huggingface.co/datasets/urecheM/llm-fpga-benchmark (also on Kaggle)
Tools	Python, VHDL-2008, GHDL, Yosys, nextpnr, SymbiYosys, pytest, Docker
Date	2026

Abstract. Hardware description code is not correct merely because it parses: it must compile under strict language semantics, synthesise to a realisable netlist and simulate correctly. This report describes hdleval, an evaluation platform for language-model-generated VHDL, consisting of a versioned 18-circuit benchmark suite, an 11-stage pipeline graded by open-source EDA tools (GHDL compilation, Yosys synthesis, GHDL simulation against oracle vectors), and statistical reporting with 95% Wilson intervals. The baseline experiment (216 runs) validates the instrument with one positive control and three graded synthetic controls. The positive control scored 54/54; the graded controls scored 72.2%, 55.6% and 44.4%, matching their designed order. All 69 failures were attributed to a single pipeline stage. Grading the same designs with static structure checks alone passed 83% to 100% of them, against a toolchain-verified rate of 44% to 72%, indicating that static-only evaluation substantially overstates HDL correctness. The baseline does not evaluate commercial language models; it establishes the measurement procedure for such studies.

Index terms: LLM code generation, VHDL, benchmark, EDA, GHDL, Yosys, Wilson interval, mutation testing, reproducibility.

1. Introduction

Language models can generate hardware description code, but assessing that code requires more than checking that it parses. A VHDL entity must pass three sequential conditions: compilation under the language semantics, synthesis to a netlist (many constructs that compile and simulate are not synthesisable), and functionally correct simulation. Evaluation that stops at syntax or structural pattern matching cannot distinguish these cases. Existing benchmarks for LLM-generated HDL [1]-[3] focus mainly on Verilog. This work targets VHDL-2008 [4] and reports compilation, synthesis, simulation and structural quality as separate stages.

The contribution is a measurement instrument rather than a ranking of models. The baseline therefore addresses three questions about the instrument itself:

- **RQ1.** Does the pipeline recover the designed quality ordering of positive and graded negative controls?
- **RQ2.** At which pipeline stage do generated designs fail?
- **RQ3.** By how much does static-only grading overstate correctness relative to toolchain-backed grading?

1.1 Contributions

1. A versioned 18-circuit VHDL benchmark released as pure data with verified reference implementations.
2. A fixed 11-stage evaluation pipeline applied identically to every (model, prompt, benchmark, trial) tuple.

3. Validation of the evaluator using positive and graded negative controls, mutation testing and bounded model checking.
4. Registry-driven artifact generation with per-run provenance and containerised execution.
5. A measured estimate of the overstatement introduced by static-only evaluation.

2. Related Work

Table 1 summarises the evaluation scope of closely related benchmarks. The distinguishing features of this work are the target language, the use of synthesis as a separate graded stage, the use of known-quality controls to validate the evaluator, and interval estimates on every reported rate.

Table 1. Evaluation scope of related LLM-for-HDL benchmarks.

Work	HDL	Evaluation basis
Thakur et al. [1]	Verilog	Compilation and functional testbenches
VerilogEval [2]	Verilog	Functional simulation against reference solutions
RTLLM [3]	Verilog	Syntax, functional simulation and synthesised design quality
This work	VHDL-2008	Compilation, synthesis, simulation, structural and resource metrics; positive and negative controls; 95% intervals

3. Benchmark Suite

The suite contains 18 circuits in seven categories (Table 2). Each benchmark specifies a prompt, functional requirements, the expected entity interface, verification properties and a verified VHDL-2008 reference implementation with oracle vectors. Difficulty is a fixed weighted sum over seven complexity dimensions; it is a property of the task and is not fitted to any model. The resulting tiers are 1 trivial, 8 easy, 6 moderate and 3 hard circuits.

Table 2. Benchmark suite v1.

Category	n	Circuits
Arithmetic	4	8-bit ripple-carry adder; 8-bit ALU (5 operations); magnitude comparator; 8x8 multiplier
Communication	3	UART transmitter (8N1); SPI master (mode 0); I2C bit-level controller
Memory	3	Synchronous FIFO (depth 16); 32x32 register file (2R1W); 256x8 RAM
FSM	2	Overlapping 1011 sequence detector; traffic-light controller
DSP	2	4-tap FIR filter; moving-average filter (window 4)
Processor	2	ALU control decoder; 2-bit saturating branch predictor
Control	2	Switch debouncer; PWM generator (8-bit duty)

4. Evaluation Methodology

The platform is layered: typed YAML configuration, a one-method ModelProvider interface (reference, synthetic or API-backed), prompt strategies, the benchmark data, toolchain adapters, metric engines, the evaluation harness, and an append-only SQLite registry from which all tables and figures are generated. Inference is decoupled from evaluation, so no code path permits model-specific grading.

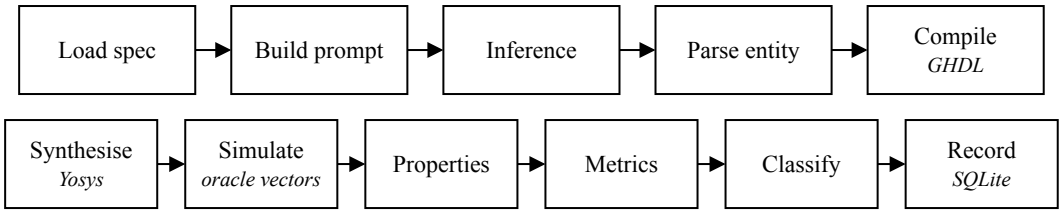


Figure 1. The 11-stage procedure applied to every (model, prompt, benchmark, trial) tuple. Stages run in order; the first failing stage determines the failure class. Absent tools are recorded as skipped, and the flag HDLEVAL_REQUIRE_TOOLS=1, used for all published results, makes a missing tool a hard error.

Metrics are recorded at four levels: functional (compile, synthesise, simulate, properties), structural (latch inference from the Yosys netlist, hierarchy depth, fan-out, FSM state count), resource (post-synthesis cell and LUT counts, estimated maximum frequency) and process (latency, tokens, retries). Each failure is assigned exactly one of nine classes by the first failing stage, so a simulation failure implies that the design compiled and synthesised. Frequency estimates are compared against nextpnr place-and-route timing on an iCE40 target, and only place-and-route figures are treated as timing measurements.

5. Experimental Design

Experiment baseline-v1 comprises 4 providers x 18 circuits x 3 seeded trials = 216 runs, with GHDL and Yosys required and their presence recorded per run. The providers are controls of known quality:

- **reference-golden** replays the verified reference implementation. It is a positive control and must score 100%.
- **synthetic-high, synthetic-mid, synthetic-low** apply seeded fault injection (dropped assignments, truncation, absent output) at three severities, giving graded negative controls with a designed quality order.

Statistical methods. Each provider outcome is a binomial proportion with n = 54, so 95% Wilson intervals [5] are used; they retain correct coverage at small n and near 0 or 1, where the normal approximation fails. Comparisons between providers on the same circuits are paired, so McNemar's test [6] is used and effect sizes are reported as Cohen's h. Per-circuit p-values are corrected with the Holm-Bonferroni procedure to control family-wise error across 18 circuits. Continuous metrics use bias-corrected and accelerated bootstrap intervals [7]. Each run records the git commit, an environment fingerprint, seeds, inference configuration and retry history.

6. Results

Table 3. Baseline-v1 results (n = 54 per provider). Source: append-only registry, GHDL and Yosys required on every run.

Provider	Compile	Simulation	95% Wilson CI	Compile failures	No output
reference-golden	100%	100%	[0.934, 1.000]	0	0
synthetic-high	72.2%	72.2%	[0.591, 0.824]	15	0
synthetic-mid	55.6%	55.6%	[0.424, 0.680]	21	3
synthetic-low	44.4%	44.4%	[0.320, 0.576]	21	9

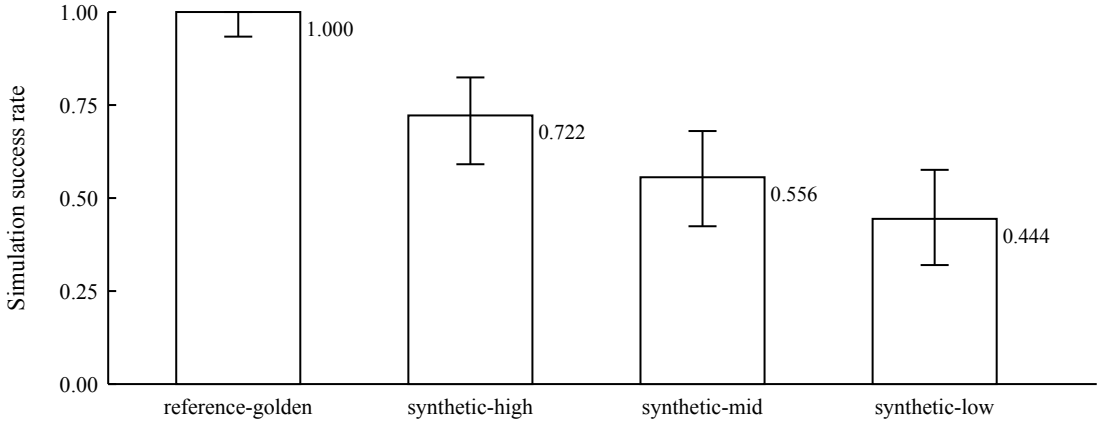


Figure 2. Simulation success rate with 95% Wilson intervals. The extreme intervals are disjoint; adjacent intervals overlap, which defines the resolution of the instrument at n = 54.

RQ1. The positive control scored 54/54 and the three graded controls ranked in their designed order. The intervals of adjacent synthetic controls overlap, so the baseline resolves the extreme quality levels but not neighbouring ones at this sample size.

7. Error Analysis

RQ2. All 69 failures fall into two classes: 57 GHDL compilation errors and 12 absent outputs. Absent outputs increase as fault severity increases (0, 3, 9). Category-level results reveal effects hidden by the aggregate: synthetic-high scored 100% on arithmetic, communication and FSM tasks but 0% on memory tasks, while synthetic-mid scored 0% on control and DSP tasks and 100% on FSM and processor tasks.

RQ3. Re-grading the same designs with static structure checks only, the documented fallback when tools are absent, passed 83% to 100% of them, compared with toolchain-verified rates of 44% to 72%. Static checks cannot determine whether code compiles or simulates, and on this data they overstate correctness by tens of percentage points.

8. Verification of the Evaluator

The evaluator is tested with the same scrutiny applied to generated designs. A seeded stuck-carry fault, not detectable by structural checks, is required to cause a GHDL simulation failure, and does. An eight-class mutation campaign [8] records which stage detects each defect class, and escapes are reported. SymbiYosys bounded model checking (depth 24) is run on emitted RTL and reported as a pass to the stated depth, not as a proof.

9. Threats to Validity

- **Construct validity.** Compilation, synthesis and simulation are necessary conditions for correct HDL but not sufficient for design quality. Named property checks and structural metrics partially address this.
- **Internal validity.** Errors in reference implementations, oracle vectors or the toolchain could bias results. Toolchain versions and the container image are hash-pinned, and the positive control tests the references and harness together.
- **External validity.** Eighteen circuits cannot represent hardware design in general; hierarchical and expert-tier designs are absent; only VHDL-2008 is covered; and the baseline uses synthetic providers rather than real models. Results should not be read as a ranking of language models.

- **Statistical validity.** Trials share circuits and are therefore not fully independent; n = 54 per provider limits resolution between neighbouring quality levels.

10. Reproducibility and Software Quality

Every number, table and figure is regenerated from the registry by *python reproduce.py*; no result is entered by hand. A Docker image ships the open-source HDL toolchain, and a VS Code dev container is provided. The test suite (66 unit, integration, regression and end-to-end tests) reaches 89% line coverage; mypy and ruff gate continuous integration, which installs the real GHDL and Yosys toolchain rather than mocks.

```
docker build -t hdleval .
docker run --rm hdleval       # runs reproduce.py end to end
pytest && coverage report     # 66 tests, 89% line coverage
```

Listing 1. Reproduction and test commands.

11. Research Outputs

Table 4. Released outputs.

Output	Status	Location
Preprint	Preprint; submitted to OpenReview	AI FPGA Engineer: Benchmarking LLM-Generated VHDL
Research software	Released v1.0 (v0.1 Prototype, v0.5 Verification)	github.com/urecheM/ai-fpga-engineer
Benchmark dataset	Public	Hugging Face: urecheM/llm-fpga-benchmark; Kaggle
Notebooks	Public (2)	Kaggle: Evaluating LLMs for Synthesizable VHDL Generation; Evaluating LLMs on Hardware Design Generation
Technical report	Prepared for Papers With Code	N/A

12. Conclusion and Future Work

The baseline shows that the pipeline recovers the designed ordering of known-quality controls, attributes every failure to a single stage, and quantifies the overstatement produced by static-only grading. Planned work includes: evaluation of real language models through the same harness; a pre-registered prompting study (zero-shot, few-shot, chain-of-thought and structured specification; 18 circuits x 10 trials; paired McNemar analysis); k-induction and equivalence checking in place of depth-bounded model checking; a second suite version with hierarchical and expert-tier tasks; and Verilog, SystemVerilog and Chisel back ends.

References

[1] S. Thakur et al., "Benchmarking large language models for automated Verilog RTL code generation," in *Proc. DATE*, 2023.

[2] M. Liu, N. Pinckney, B. Khailany and H. Ren, "VerilogEval: Evaluating large language models for Verilog code generation," in *Proc. ICCAD*, 2023.

[3] Y. Lu, S. Liu, Q. Zhang and Z. Xie, "RTLLM: An open-source benchmark for design RTL generation with large language model," in *Proc. ASP-DAC*, 2024.

[4] *IEEE Standard VHDL Language Reference Manual*, IEEE Std 1076-2008, 2009.

[5] E. B. Wilson, "Probable inference, the law of succession, and statistical inference," *J. Amer. Statist. Assoc.*, vol. 22, no. 158, pp. 209-212, 1927.

[6] Q. McNemar, "Note on the sampling error of the difference between correlated proportions or percentages," *Psychometrika*, vol. 12, no. 2, pp. 153-157, 1947.

[7] B. Efron and R. J. Tibshirani, *An Introduction to the Bootstrap*. Chapman and Hall, 1993.

[8] Y. Jia and M. Harman, "An analysis and survey of the development of mutation testing," *IEEE Trans. Softw. Eng.*, vol. 37, no. 5, pp. 649-678, 2011.

[9] T. Gingold et al., GHDL: open-source VHDL analyser, compiler and simulator. github.com/ghdl/ghdl

[10] C. Wolf et al., Yosys Open SYnthesis Suite. yosyshq.net/yosys